# 정세진 Backend Developer

## 데이터 정합성을 검증하고 처리 구조를 개선하는 백엔드 개발자

Java·Spring Boot 기반 수강 신청 시스템에서 조건부 UPDATE로 동시성 문제를 해결하고, 부하 테스트를 통해 정원 초과와 중복 신청이 발생하지 않음을 검증했습니다. 또한 Kafka 기반 비동기 처리에서 재시도·DLT와 메시지 중복 반영 방지를 구현했으며, 서비스 간 API 연동과 CI/CD 자동화까지 경험했습니다.


GitHub
https://github.com/sejinzx

Email
jeongsejin789@gmail.com

블로그
https://vivid-reminiscences.tistory.com/


### 보유기술

**Core**

Java | Spring Boot | MySQL

**Experienced**

Apache Kafka | Spring Security | Redis | Docker | GitHub Actions | AWS EC2

**Collaboration & Tools**

GitHub | Notion | Slack

PROJECT 01

# 클래스룸

## 프로젝트 개요

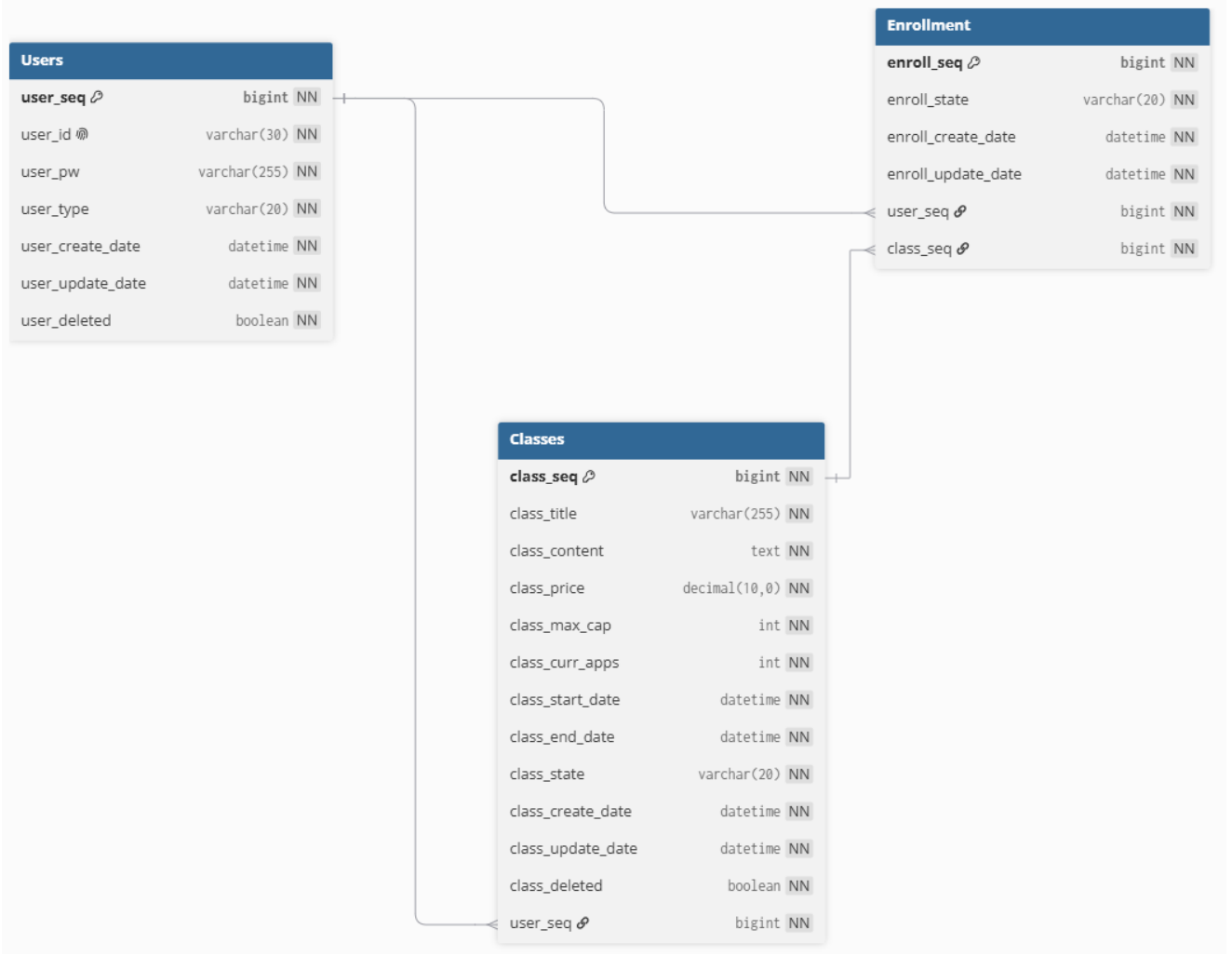
사용자가 강의를 조회하고 수강 신청·취소할 수 있는 웹 서비스입니다.
다수의 사용자가 동시에 신청하는 환경에서 정원 초과와 중복 신청을 방지하고, 비동기 메시지 처리 과정에서도 데이터 정합성을 유지하도록 설계했습니다.

- 기간: 2026.05~09
- 형태: 개인 프로젝트
- 기술스택: Java, Spring Boot, MySQL, Apache Kafka, Docker, GitHub Actions, AWS, JMeter, Testcontainers
- GitHub: https://github.com/sejinzx/enrollment_system
- Web: http://13.209.75.120:3000

## 주요 구현

- Kafka Producer–Consumer 기반 비동기 수강 신청
- 조건부 UPDATE를 활용한 동시성 제어
- DB UNIQUE 제약 및 트랜잭션을 활용한 데이터 정합성 확보
- Kafka Retry·DLT 기반 메시지 장애 처리
- JUnit·Testcontainers 기반 동시성 및 통합 테스트
- GitHub Actions 기반 CI/CD 및 AWS EC2 배포

## ERD




## 조건부 UPDATE를 적용한 동시 요청 데드락 해결 https://buly.kr/YhaAA3

### 문제 및 원인

비관적 락 기반 수강 신청 처리 과정에서 동시 요청 증가로 동일 강의 데이터에 대한 락 충돌과 데드락이 발생했습니다.

### 해결 과정

비관적 락에서 발생한 락 경쟁을 줄이기 위해 정원 확인과 신청 인원 증가를 조건부 UPDATE로 통합했습니다. UPDATE 결과로 정원 확보 여부를 판단하고, UNIQUE 제약 및 트랜잭션을 통해 중복 신청을 방지했습니다.

```
UPDATE classes
SET class_curr_apps = class_curr_apps + 1
WHERE class_seq = ?
  AND class_deleted = false
  AND class_state = 'OPEN'
  AND class_curr_apps < class_max_cap;
```

PROJECT 01

# 클래스룸

### 검증 결과

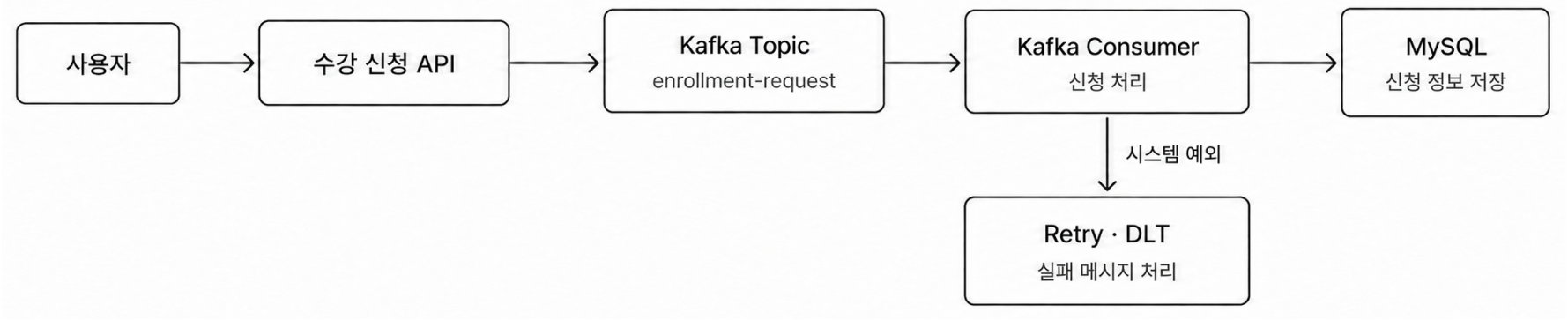
테스트 환경: JUnit 5·Testcontainers 기반 MySQL·Kafka 통합 테스트

| 테스트 조건 | 결과 |
| --- | --- |
| 정원 100명 / 동시 신청 200명 | 성공 100건, 정원 초과 100건 |
| 동일 사용자 동시 신청 20회 | 실제 신청 데이터 1건 |
| 데이터 정합성 | 신청 인원과 실제 신청 건수 일치 |

## Kafka 기반 비동기 수강 신청 처리

### 아키텍처 및 기술 선택

수강 신청 요청 접수와 실제 DB 처리를 분리하여 수강 신청을 비동기로 처리하기 위해 Kafka를 도입했습니다.




### 장애 처리 및 데이터 정합성

- 정원 초과·중복 신청 등 비즈니스 예외는 재시도하지 않음
- 시스템 예외는 최대 2회 재시도하고, 최종 실패 시 DLT로 전달
- UNIQUE 제약과 트랜잭션으로 메시지 중복 전달에 따른 중복 반영 방지

### 통합 테스트

| 테스트 | 검증 결과 |
| --- | --- |
| 동일 Kafka 메시지 2회 전달 | 신청 데이터 1건만 반영 |
| 일시적 시스템 장애 | Retry 후 정상 처리 |
| 재시도 최종 실패 | DLT 전달 |

## Codex를 활용한 프론트엔드 구현

직접 개발한 Spring Boot 백엔드 코드와 API 구조를 기반으로 Codex를 활용해 프론트엔드를 구현했습니다.

PROJECT 02

# Talk-kit

## 프로젝트 개요

AI 대본 생성, 음성 녹음, 프로젝트 관리 및 커뮤니티 피드백 기능을 제공하는 AI 기반 발표 연습 플랫폼입니다.
Spring Cloud Gateway와 Eureka를 활용해 기능별 서비스를 분리한 MSA 구조로 개발했습니다.
팀 구성 변경 이후 유저·커뮤니티 서비스의 잔여 기능을 담당하고 서비스 간 데이터 연동 작업을 수행했습니다.

- 기간: 2024.06~10
- 인원: Backend 3명 / Frontend 2명
- 담당 역할: 유저 서비스, 커뮤니티 서비스 개발
- 기술스택: Java, Spring Boot, Spring Cloud Gateway, Eureka, MySQL, Redis, JWT, Docker, Kubernetes, NHN Cloud
- GitHub: https://github.com/orgs/Talk-Kit/repositories

## 담당 기능

- 유저·커뮤니티 서비스 API 개발
- Redis TTL을 활용한 이메일 인증번호 저장 및 30분 만료 처리
- Feign Client를 활용한 서비스 간 데이터 연동

## 서비스 간 데이터 연동 구조 구현

### 문제 및 원인

커뮤니티 게시글에서 프로젝트에 저장된 파일을 활용하기 위해 프로젝트 서비스의 파일 정보를 조회하는 서비스 간 통신이 필요했습니다.

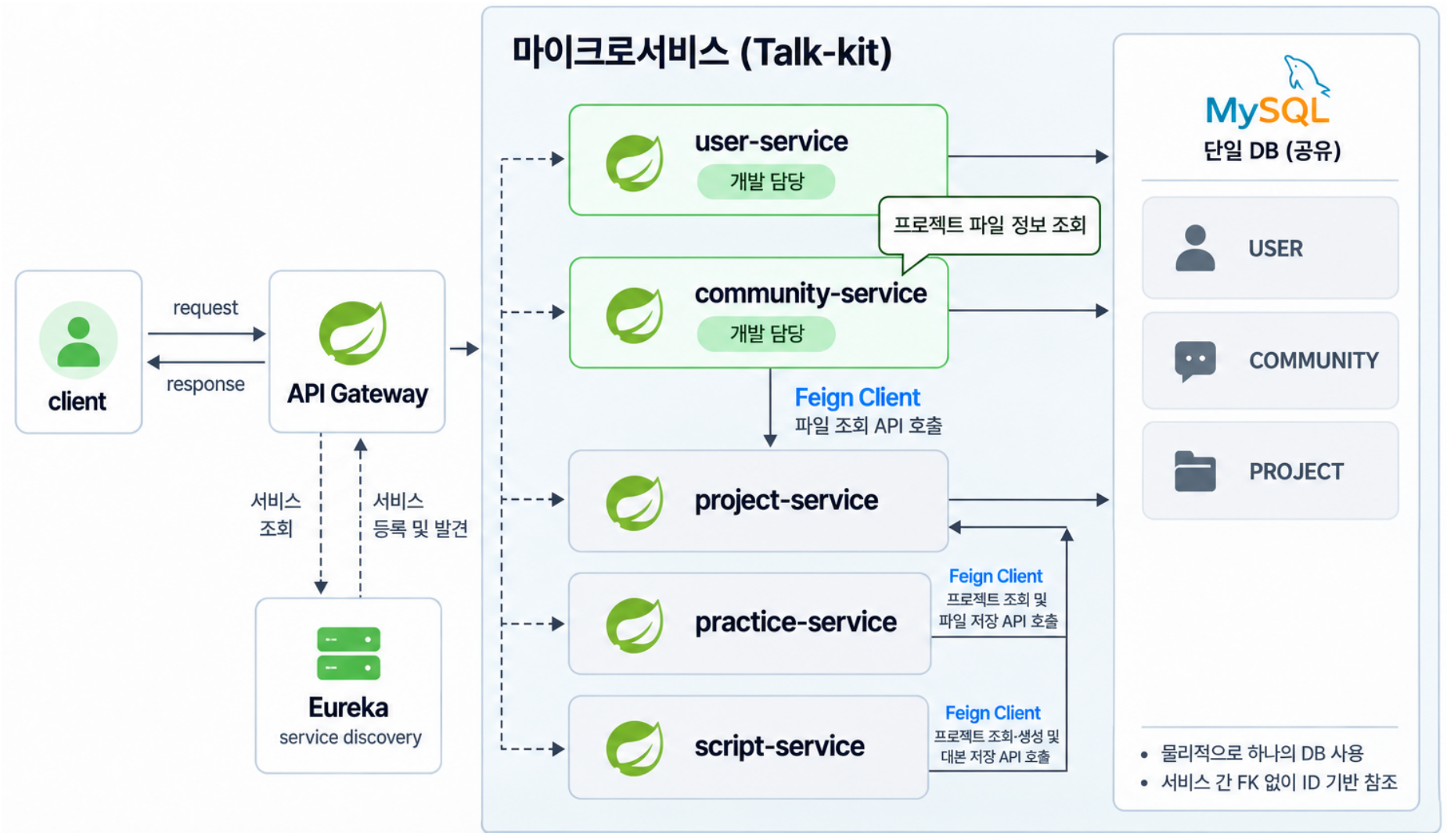
### 해결 과정

Feign Client를 활용해 프로젝트 서비스의 파일 조회 API를 연동하고, RequestInterceptor를 통해 JWT 인증 헤더를 전달하도록 구현했습니다.

### 결과

커뮤니티 서비스에서 프로젝트 파일 정보를 API로 조회할 수 있도록 구성하여 서비스 간 직접적인 데이터 접근을 분리했습니다.

## 시스템 아키텍처



PROJECT 03

# 일정 관리 시스템 (Scheduling)

## 프로젝트 개요

React와 Spring Boot 기반의 일정 및 Todo 관리 웹 애플리케이션입니다.
Spring Boot, React, MySQL을 Docker 컨테이너로 구성하고 Jenkins를 활용해 빌드 및 실행 과정을 자동화했습니다.

- 기간: 2026.04~06
- 형태: 개인 프로젝트
- 기술스택: Java, Spring Boot, React, MySQL, Docker, Jenkins
- GitHub: https://github.com/sejinzx/scheduling

## Docker 기반 실행 환경 및 Jenkins 빌드·실행 자동화

- API 통신: 컨테이너 내부 localhost가 자기 자신을 가리키는 문제를 해결하기 위해 React 프록시를 spring-app:8080으로 변경
- 빌드 구조: Multi-stage Build로 Gradle 빌드 단계와 JAR 실행 단계를 분리
- 자동화: Jenkins Pipeline에서 GitHub main 브랜치를 가져온 뒤 docker compose down → docker compose up --build -d 실행

## 시스템 아키텍처

